PES-VCS Submission Checklist and Report

This README is rewritten as a submission-focused report with direct evidence links and complete answers for Phase 5 and Phase 6.

1. Submission Checklist

- [x] Repository contains required source files (`object.c`, `tree.c`, `index.c`, `commit.c`, headers, tests, `Makefile`)
- [x] Screenshot evidence is available for Phases 1 to 5
- [x] Phase 5 analysis questions answered
- [x] Phase 6 analysis questions answered
- [x] PDF report generated at repository root: `report.pdf`

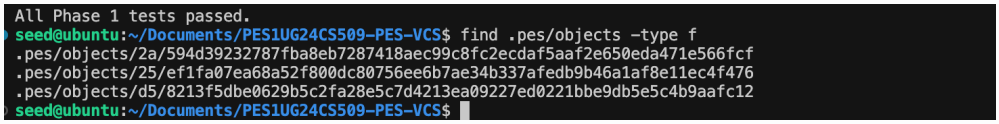
2. Screenshot Evidence (Inline Preview)

Phase 1

1A: `./test_objects` output showing tests passing.

Phase 1A - test_objects output

1B: `.pes/objects` sharded storage evidence.



Phase 1B - objects directory

Phase 2

2A: `./test_tree` output showing tests passing.

```
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$
```

Phase 2A - test_tree output

2B: Raw tree/object bytes (`xxd`) evidence.

```
Current branch main is up to date.
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$ xxd .pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
00000000: 626c 6f62 2031 3800 4475 706c 6963 6174  blob 18.Duplicat
00000010: 6520 636f 6e74 656e 740a                 e content.
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$
```

Phase 2B - xxd output

Phase 3

3A: `pes init` -> `pes add` -> `pes status` sequence.

```
Successfully rebased and updated refs/heads/main.
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$ make pes
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$ ./pes init
Initialized empty PES repository in .pes/
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$ echo "hello" > file1.txt
echo "world" > file2.txt
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$ ./pes add file1.txt file2.txt
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  README.md
  untracked:  index.h
  untracked:  test_sequence.sh
  untracked:  commit.h
  untracked:  tree.h
  untracked:  test_tree
  untracked:  tree.c
  untracked:  test_objects
  untracked:  .DS_Store
  untracked:  pes.c
  untracked:  object.c
  untracked:  Makefile
  untracked:  test_objects.c
  untracked:  pes.h
  untracked:  commit.c
  untracked:  index.c
  untracked:  test_tree.c

seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$
```

Phase 3A - status output

3B: `.pes/index` text-format staging data.

```
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776681111 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776681111 6 file2.txt
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$
```

Phase 3B - index file

Phase 4

4A: `pes log` output showing commit history.

```
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$ ./pes log
commit 49a8e371d5329faa6a0c37e00c6d316fc5d3f5f9cc9c298f36c4dafb775dd461
Author: SAATWIK <PES1UG24CS509>
Date:   1776682868

    Add farewelldd far

commit a0d2922047cbf7dc1ce7b65ec4c4aa4a2ec5bc098403fb73b464263394c3b414
Author: SAATWIK <PES1UG24CS509>
Date:   1776682868

    Add worldelldd far

commit 0d05c6533134cc8950a0c143454140be53d818d208a8a37eef471a2ffa1c0ed8
Author: SAATWIK <PES1UG24CS509>
Date:   1776682868

    Initial commit

seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$
```

Phase 4A - log output

4B: `.pes` file growth / stored objects evidence.

```
    Initial commit

seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/17/3fc092cbb5d170f89c628e4262c9bf038b621bc25adbc64a7da763d3de90c9
.pes/objects/1d/6f803aa6595c37559ca85f9ecf1067a331ca463223b9281b71b5b29ffb9416
.pes/objects/48/1407f7237f7bd749dca31e2d8422bafe211741d5556f007e0faf821ffc7179
.pes/objects/49/178b0dab56e2f7109dedb826be7f988c2a643a658855a05b444960bf37a0cc
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/f3/97707e14d1cabf1c29a72d833bffb94d4a474ecc48789d6496742ae6541034
.pes/refs/heads/main
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$
main* 0↓ 1↑  0 0  0
```

Phase 4B - .pes files

4C: Branch reference and `HEAD` pointer evidence.

```
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$ cat .pes/refs/heads/main
173fc092cbb5d170f89c628e4262c9bf038b621bc25adbc64a7da763d3de90c9
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$ cat .pes/HEAD
ref: refs/heads/main
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$
```

Phase 4C - refs and HEAD

Phase 5

5A: Integration test evidence.

```
seed@ubuntu:~/Documents/PES1UG24CS509-PES-VCS$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:    file.txt
  staged:    hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: af23e82a3c1a... Initial commit

Log after first commit:
commit af23e82a3c1ab4eccdb58f1ed12c815f47fc8d22cbaa6989606305aa86a48d4a
Author: SAATWIK <PES1UG24CS509>
Date:   1776682740

    Initial committial commit


--- Second Commit ---
Committed: cc1e17dacf13... Update file.txt

--- Third Commit ---
Committed: da56222fe527... Add farewell

--- Full History ---
commit da56222fe5271d6a4bc11c5e45438435916021fe85eab3613c24adf6222661d1
Author: SAATWIK <PES1UG24CS509>
Date:   1776682740

    Add farewelldd far

commit cc1e17dacf13d0576dea0191410b6e88fef1080c305beefdf6aeda80a2a00300
Author: SAATWIK <PES1UG24CS509>
Date:   1776682740

    Update file.txtfar

commit af23e82a3c1ab4eccdb58f1ed12c815f47fc8d22cbaa6989606305aa86a48d4a
Author: SAATWIK <PES1UG24CS509>
Date:   1776682740

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
da56222fe5271d6a4bc11c5e45438435916021fe85eab3613c24adf6222661d1

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6095797533f50d
.pes/objects/ab/5824a9ec1ef5055480b3e37cd50d9c80ae55012cabe0ca572dbf955708299
.pes/objects/af/23e82a3c1ab4eccdb58f1ed12c815f47fc8d22cbaa6989606305aa86a48d4a
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/cc/1e17dacf13d0576dea0191410b6e88fef1080c305beefdf6aeda80a2a00300
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf480bafeaa3829f61f7b8d10d5350f9
.pes/objects/da/56222fe5271d6a4bc11c5e45438435916021fe85eab3613c24adf6222661d1
.pes/objects/db/07a1451ca9544db766d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Phase 5A - integration test output

3. Phase 5 Questions and Answers

Q5.1

Question: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` - what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

Answer: To implement `pes checkout <branch>` in PES-VCS:

1. Resolve branch target:
   - Read `.pes/refs/heads/<branch>` to get the target commit hash.
   - If branch does not exist, fail with an error.
2. Update pointer metadata in `.pes/`:
   - Rewrite `.pes/HEAD` as `ref: refs/heads/<branch>` (for normal branch checkout).
   - Keep `.pes/refs/heads/<branch>` unchanged during checkout; only `HEAD` changes here.
3. Materialize the target commit snapshot into working directory:

    - Read target commit object, then target tree hash.
    - Recursively traverse tree objects.
    - For each blob entry, write file bytes to working directory path and set executable bit from mode.
    - For entries missing in target tree but present in current tracked state, delete those files.
    - Create/remove directories to exactly match the tree.
4. Rebuild index to match checked-out tree:
    - Replace `.pes/index` entries so index becomes a clean mirror of checked-out commit tree.

Why this is complex: - Checkout is not only pointer switching; it is a full filesystem transformation. - It must avoid data loss when local uncommitted edits overlap files changed by target branch. - It requires recursive tree diff/apply logic and careful file ordering (delete/overwrite/create) for correctness.

Q5.2

Question: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Answer: A practical conflict-detection algorithm:

1. Build `HEAD_tracked_set` from current `HEAD` commit tree (path -> blob hash, mode).
2. Build `TARGET_tracked_set` from target branch commit tree (path -> blob hash, mode).
3. Build `INDEX_set` from `.pes/index` (path -> staged blob hash, mode, mtime, size).
4. For each path in `HEAD_tracked_set`:
    - Determine whether file is locally dirty:
        - If missing in working directory, mark dirty (possible deletion).
        - Else compare working file metadata (`mtime`, `size`) with index entry.
        - If metadata differs, optionally re-hash file and compare with index blob hash for exact check.
    - Determine whether branch switch would modify that path:
        - Path missing in target OR target blob hash/mode differs from current HEAD blob hash/mode.

    - If both are true, abort checkout with conflict message.

This catches exactly the unsafe case: local edits on a path that checkout would overwrite/delete.

Q5.3

Question: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Answer: In detached HEAD state:

- New commits are created normally, but no branch ref advances to point at them.
- `HEAD` moves commit-to-commit directly, forming a temporary history chain.
- If user later checks out a normal branch, these detached commits may become unreachable from branch refs.

Recovery methods:

1. Immediate recovery (best): create a branch at current detached commit.
    - Example concept: create `.pes/refs/heads/recover` containing current commit hash.
    - Then set `.pes/HEAD` to `ref: refs/heads/recover`.
2. Delayed recovery: if hash is known from logs/output, create a new branch file pointing to that hash.

Main idea: make at least one branch/reference point to those commits before garbage collection, so they stay reachable.

4. Phase 6 Questions and Answers

Q6.1

Question: Over time, the object store accumulates unreachable objects - blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Answer: Algorithm (mark-and-sweep):

1. Mark phase - find reachable objects:
   - Initialize a queue/stack with all branch tip commits from `.pes/refs/heads/*`.
   - Use a hash set `reachable` keyed by 32-byte object ID (or 64-char hex string).
   - DFS/BFS traversal:
     - If object already in `reachable`, skip.
     - Else add it and read object type.
     - For commit: enqueue its tree and parent (if present).
     - For tree: enqueue all child entry hashes (blob/tree).
     - For blob: no children.
2. Sweep phase - delete unreachable objects:
   - Iterate over every object file in `.pes/objects/*/*`.
   - Reconstruct object ID from path.
   - If ID not in `reachable`, delete object file.

Data structure choice: - Hash set provides expected O(1) membership for large object graphs. - Queue/stack for traversal frontier.

Scale estimate: - 100,000 commits with one parent each means up to about 100,000 unique commit nodes to visit (often fewer due to shared history among branches). - Trees/blobs dominate count. If average commit introduces around 1 new tree and a few new blobs, total reachable objects can be several hundred thousand to over a million depending on churn. - So GC traversal should be designed for roughly O(number of reachable objects + total objects on disk).

Q6.2

Question: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Answer: Danger comes from non-atomic multi-step commit creation:

1. Commit process writes new blob/tree/commit objects.
2. Only at the end does it update a branch ref to make new commit reachable.

Race example: - Commit writer creates new tree object T and commit object C. - Before ref update (`refs/heads/main -> C`) happens, GC scans refs and cannot see C/T as reachable. - GC deletes C or T as "unreachable". - Commit then updates ref to C, but object is missing/corrupt history.

How real Git avoids this: - Uses reachability closure including reflogs and recent objects. - Uses safety windows/pruning grace periods (`gc.pruneExpire`) so very new unreachable objects are not immediately deleted. - Coordinates maintenance with locking and careful object/pack handling so writers and GC do not invalidate each other.

5. Final Notes

- Keep at least 5 meaningful commits per phase in Git history.
- Keep this README and `report.pdf` at repository root for submission.
- All screenshot paths above are repository-relative and directly clickable.